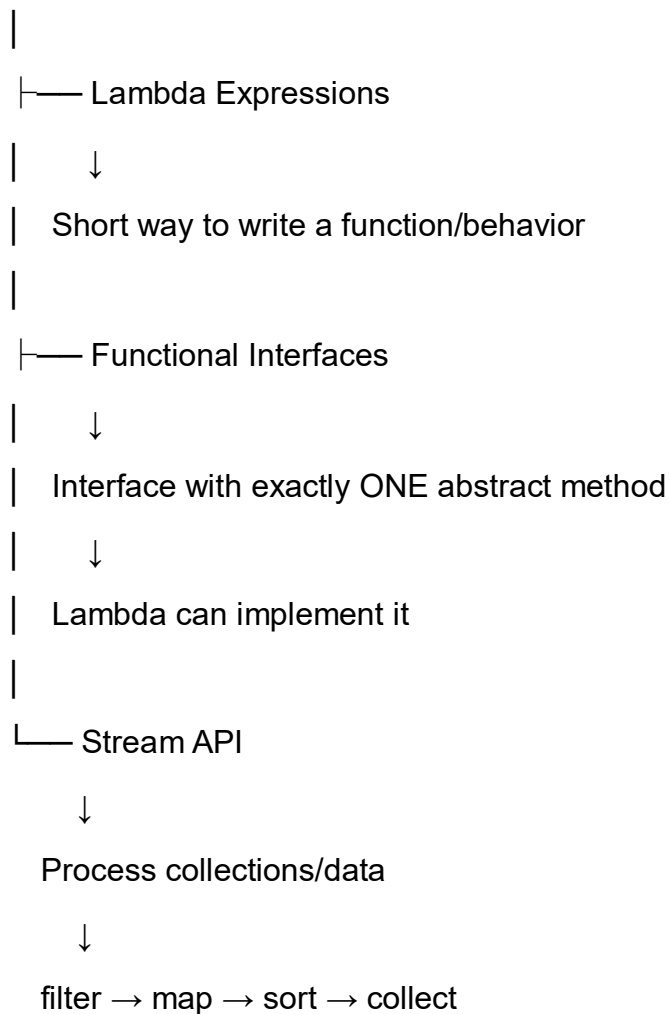


Day 38: Java 8 Features

- Lambda expressions
- Functional interfaces
- Stream API

Java 8



1. First: Why did Java 8 introduce these features?

Old Java code sometimes **too much lengthy** and **undecipherable**.

For example, suppose you have a list of numbers like:

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);
```

Suppose manaki each number print cheyyali.

Old style:

```
for (Integer number : numbers) {  
    System.out.println(number);  
}
```

Idi perfectly valid.

But Java 8 taruvatha:

```
numbers.forEach(number -> System.out.println(number));
```

Inka simple ga:

```
numbers.forEach(System.out::println);
```

Java 8 main idea:

"What behavior do you want?" ane concept ni easy ga express cheyyadam.

Three major concepts:

- Lambda → behavior ni short ga write cheyyadaniki
- Functional Interface → aa behavior ni represent cheyyadaniki
- Stream → data ni process cheyyadaniki

PART 1 — Lambda Expressions

2. Lambda expression ante enti?

Simple definition:

Lambda expression is a short way of writing an anonymous function/behavior in Java.

But ikkada "function" word chusi confuse avvakudadhu.

Java lo normal method:

```
public void add() {  
    System.out.println("Hello");  
}
```

Lambda:

```
() -> System.out.println("Hello")
```

Lambda ki:

(parameters) -> expression

or

(parameters) -> {
 statements
}

structure untundi.

3. Lambda syntax

Basic syntax:

(parameters) -> expression

Example:

(int a, int b) -> a + b

Meaning:

a and b inputs

↓

a + b

↓

output

Another:

() -> System.out.println("Hello")

Meaning:

No input

↓

print Hello

Another:

name -> System.out.println(name)

Meaning:

Take name

↓

print name

4. Lambda lo -> meaning enti?

-> ni **arrow operator** / lambda operator ani antaru.

Example:

$x \rightarrow x * 2$

Read it like:

"Take x, then return $x * 2$."

So:

$x \rightarrow x * 2$

is conceptually:

input x

↓

$x * 2$

↓

result

5. Lambda expressions examples

No parameters

() -> System.out.println("Hello");

One parameter

x -> System.out.println(x);

Two parameters

(a, b) -> a + b

Multiple statements

(a, b) -> {

int sum = a + b;

return sum;

```
}
```

Important:

If {} use chesthe, return explicitly rayali when you need a returned value.

```
(a, b) -> {  
    return a + b;  
}
```

Without braces:

```
(a, b) -> a + b  
automatic expression result return avutundi.
```

6. Lambda itself complete program kaadu

This is a **very important point**.

Nuvvu ila rayav:

```
x -> x * 2
```

Java ki question:

"x enti? Ee lambda ni ekkada use cheyyali?"

Compiler ki context kavali.

For example:

```
Function<Integer, Integer> doubleNumber = x -> x * 2;
```

Ikkada:

```
Function<Integer, Integer>
```

↓

Lambda expected behavior

↓

```
x -> x * 2
```

So lambda usually **functional interface context lo** use avutundi.

PART 2 — Functional Interfaces

7. Interface ante enti?

Interface concept already OOP lo chusam.

Example:

```
interface Animal {  
    void sound();  
}
```

Animal interface lo:

```
void sound();
```

abstract method undi.

Class implement chestundi:

```
class Dog implements Animal {
```

```
    public void sound() {  
        System.out.println("Bark");  
    }  
}
```

But Java 8 introduced a special concept:

Functional Interface

Definition:

An interface that contains exactly one abstract method is called a Functional Interface.

Example:

```
@FunctionalInterface  
interface Greeting {  
    void sayHello();  
}
```

Only one abstract method:

```
void sayHello();
```

Therefore it is a functional interface.

8. Why exactly ONE abstract method?

Because lambda represents **one behavior**.

Suppose:

```
interface Calculator {  
    int calculate(int a, int b);  
}
```

Only one abstract method.

So:

Calculator c = (a, b) -> a + b;

Java understands:

Calculator

↓

calculate()

↓

(a,b) -> a+b

But imagine:

```
interface Calculator {  
    int add(int a, int b);  
    int subtract(int a, int b);  
}
```

Now:

Calculator c = (a, b) -> a + b;

Java asks:

Ee lambda add() kosama?

subtract() kosama?

Ambiguous.

That's why functional interface should have **exactly one abstract method**.

9. @FunctionalInterface

We can write:

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

@FunctionalInterface is an annotation.

It tells compiler:

"I want this interface to be a functional interface."

If accidentally two abstract methods add chesthe:

```
@FunctionalInterface
interface Calculator {

    int calculate(int a, int b);

    int anotherMethod();
}
```

Compiler error vastundi.

So annotation gives us a safety check.

10. Lambda + Functional Interface connection

This is the most important connection.

Suppose:

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

Now:

Calculator c = (a, b) -> a + b;

Let's understand line by line.

Step 1

Calculator c

c variable type is Calculator.

Step 2

(a, b) -> a + b

This is lambda.

Step 3

Calculator has:

```
int calculate(int a, int b);
```

Therefore Java understands:

a → first parameter

b → second parameter

a + b → return value

So effectively:

Calculator c = (a, b) -> a + b;

means:

"Create a Calculator implementation whose calculate() behavior is a + b."

Then:

```
System.out.println(c.calculate(10, 20));
```

Output:

30

11. What actually happens behind the scenes?

Before Java 8, you might write:

```
class MyCalculator implements Calculator {
```

```
    public int calculate(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
}
```

Then:

```
Calculator c = new MyCalculator();
```

Lots of code.

Lambda:

```
Calculator c = (a, b) -> a + b;
```

Much shorter.

Conceptually:

Traditional

Interface



Implementation class



Override method



Object



Call method

Lambda

Functional Interface



Lambda behavior



Use it

Lambda doesn't mean Java suddenly stopped using objects/interfaces. It gives a concise way to provide the implementation of the functional interface's single abstract method.

12. Built-in Functional Interfaces

Java already provides many functional interfaces inside:

`java.util.function`

The four most important ones for interviews are:

Predicate

Function

Consumer

Supplier

13. Predicate

Predicate means:

Takes input and returns true/false.

Structure:

`Predicate<T>`

Its method:

`boolean test(T t);`

Example:

`Predicate<Integer> even = x -> x % 2 == 0;`

Now:

`System.out.println(even.test(10));`

Output:

true

And:

`System.out.println(even.test(7));`

Output:

false

Think:

Input



Predicate



true / false

Real-world:

Age = 20



Is age >= 18?



true

14. Function

Function means:

Takes one input and produces one output.

Structure:

Function<T, R>

T = input type

R = result/output type

Example:

Function<Integer, Integer> square = x -> x * x;

Then:

System.out.println(square.apply(5));

Output:

25

Flow:

5



square function

↓

25

Another:

```
Function<String, Integer> length = str -> str.length();
```

```
System.out.println(length.apply("Java"));
```

Output:

4

15. Consumer

Consumer:

Takes input but does not return a result.

Structure:

Consumer<T>

Method:

```
void accept(T t);
```

Example:

```
Consumer<String> print = name -> System.out.println(name);
```

Use:

```
print.accept("Java");
```

Output:

Java

Flow:

Input

↓

Consumer

↓

Action

↓

No return

For example:

```
Consumer<Integer> display = x -> System.out.println(x);
```

16. Supplier

Supplier:

Takes no input but supplies/returns a value.

Structure:

```
Supplier<T>
```

Method:

```
T get();
```

Example:

```
Supplier<Double> randomValue = () -> Math.random();
```

Then:

```
System.out.println(randomValue.get());
```

Flow:

No input

↓

Supplier

↓

Value

17. Remember these four

Interface	Input	Output	Method
-----------	-------	--------	--------

Predicate	Yes	boolean	test()
-----------	-----	---------	--------

Function	Yes	value	apply()
----------	-----	-------	---------

Consumer	Yes	nothing	accept()
----------	-----	---------	----------

Supplier	No	value	get()
----------	----	-------	-------

Easy memory trick:

Predicate → "Is it true?"

Function → "Transform it"

Consumer → "Use it"

Supplier → "Give me something"

PART 3 — Stream API

Now main topic.

18. Stream ante enti?

First, **Stream API is NOT a data structure.**

Very important.

ArrayList is a collection/data structure.

Stream is a way to **process data**.

Example:

```
List<Integer> numbers =
```

```
    Arrays.asList(10, 20, 30, 40, 50);
```

List stores data.

Stream processes that data:

```
numbers.stream()
```

Think:

Collection

↓

Stream

↓

Process data

19. Real-world example

Imagine supermarket lo:

100 products

You want:

Price > ₹1000 products only.

Traditional loop:

```
for (Product p : products) {
```

```
    if (p.price > 1000) {
```

```
        System.out.println(p);
```

```
    }
```

```
}
```

Stream:

```
products.stream()
```

```
    .filter(p -> p.price > 1000)
```

```
    .forEach(System.out::println);
```

This style is called **declarative style**.

You are saying:

"I want products whose price > 1000."

Rather than manually controlling every iteration.

20. Stream pipeline

A Stream generally follows:

Source

↓

Intermediate Operations

↓

Terminal Operation

Example:

```
numbers.stream()
```

```
    .filter(x -> x > 20)
```

```
    .map(x -> x * 2)
```



```
.forEach(System.out::println);
```

Break it:

numbers

↓

stream()

↓

filter()

↓

map()

↓

forEach()

21. Stream Source

Stream can come from collections.

```
List<Integer> numbers =
```

```
    Arrays.asList(10, 20, 30);
```

```
numbers.stream();
```

Array:

```
Arrays.stream(numbersArray);
```

You can also create streams using:

```
Stream.of(10, 20, 30);
```

22. stream()

Example:

```
List<Integer> numbers =
```

```
    Arrays.asList(10, 20, 30, 40);
```

```
numbers.stream();
```

This doesn't immediately process all numbers.

It creates a stream pipeline that can be processed by subsequent operations.

23. Intermediate operations

Intermediate operations:

Stream ni transform/filter chestayi and usually another Stream return chestayi.

Important intermediate operations:

`filter()`

`map()`

`sorted()`

`distinct()`

`limit()`

`skip()`

Let's learn each.

24. `filter()`

`filter()` means:

Condition satisfy ayye elements matrame select cheyyi.

Example:

`List<Integer> numbers =`

`Arrays.asList(10, 15, 20, 25, 30);`

`numbers.stream()`

`.filter(x -> x > 20)`

`.forEach(System.out::println);`

Output:

25

30

Flow:

10 → false → reject

15 → false → reject

20 → false → reject

25 → true → keep

30 → true → keep

Notice:

$x \rightarrow x > 20$

is a **Predicate**.

Because it returns:

true / false

25. map()

map() means:

Every element ni transform/change cheyyi.

Example:

List<Integer> numbers =

Arrays.asList(1, 2, 3, 4);

numbers.stream()

.map(x -> x * 2)

.forEach(System.out::println);

Output:

2

4

6

8

Flow:

1 → 2

2 → 4

$3 \rightarrow 6$

$4 \rightarrow 8$

map() usually takes a **Function**.

Because:

input $\rightarrow$ output

26. filter() vs map()

Very important interview question.

filter

`.filter(x -> x > 10)`

Purpose:

Select

Number of elements can reduce.

Example:

10

20

30

40

after filter:

20

30

40

map

`.map(x -> x * 2)`

Purpose:

Transform

Example:

10

20

30

becomes:

20

40

60

Easy:

filter → WHICH elements?

map → WHAT should each element become?

27. sorted()

Sort stream elements.

List<Integer> numbers =

Arrays.asList(40, 10, 30, 20);

numbers.stream()

.sorted()

.forEach(System.out::println);

Output:

10

20

30

40

Descending:

numbers.stream()

.sorted((a, b) -> b - a)

.forEach(System.out::println);

28. distinct()

Removes duplicate elements.

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 10, 30, 20);
```

```
numbers.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Output:

```
10  
20  
30
```

29. limit()

Only first N elements.

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30, 40, 50);
```

```
numbers.stream()  
    .limit(3)  
    .forEach(System.out::println);
```

Output:

```
10  
20  
30
```

30. skip()

Skip first N elements.

```
numbers.stream()  
    .skip(2)  
    .forEach(System.out::println);
```

If:

10 20 30 40 50

skip 2:

30 40 50

31. Terminal Operations

Terminal operation:

Stream processing ni finish chestundi and final result/action istundi.

Important terminal operations:

`forEach()`

`collect()`

`count()`

`reduce()`

`min()`

`max()`

`anyMatch()`

`allMatch()`

`noneMatch()`

32. `forEach()`

Most basic.

`numbers.stream()`

`.forEach(x -> System.out.println(x));`

Short:

`numbers.stream()`

`.forEach(System.out::println);`

`forEach()` is a terminal operation.

33. `collect()`

Very important.

Suppose:

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30, 40);
```

Filter:

```
List<Integer> result =  
    numbers.stream()  
        .filter(x -> x > 20)  
        .collect(Collectors.toList());
```

Now:

numbers

↓

10 20 30 40

↓ filter >20

30 40

↓

collect

↓

List

collect() stream result ni collection ga convert cheyyadaniki commonly use chestam.

34. count()

Count elements.

long count =

```
    numbers.stream()  
        .filter(x -> x > 20)  
        .count();
```

If:

30

40

then:

count = 2

Important:

count() returns:

long

not int.

35. min() and max()

Example:

Optional<Integer> min =

```
numbers.stream().min(Integer::compareTo);
```

Maximum:

Optional<Integer> max =

```
numbers.stream().max(Integer::compareTo);
```

Why Optional?

Because collection might be empty.

Suppose:

[]

minimum value exists aa?

No.

So Java uses:

Optional

to represent:

Value exists

or

Value may not exist

36. anyMatch()

Checks:

At least one element condition satisfy chestunda?

boolean result =

```
numbers.stream()  
    .anyMatch(x -> x > 30);
```

If numbers:

10 20 30 40

40 > 30.

So:

true

37. allMatch()

Checks:

Every element condition satisfy chestunda?

boolean result =

```
numbers.stream()  
    .allMatch(x -> x > 0);
```

For:

10 20 30

all positive.

Result:

true

38. noneMatch()

Checks:

No element condition satisfy chestunda?

boolean result =

```
numbers.stream()  
    .noneMatch(x -> x < 0);
```

If all are positive:

true

39. reduce()

This is slightly advanced but important.

reduce() means:

Multiple elements ni combine chesi one final value create cheyyadam.

Example sum:

```
List<Integer> numbers =
```

```
    Arrays.asList(10, 20, 30, 40);
```

```
int sum =
```

```
    numbers.stream()
```

```
        .reduce(0, (a, b) -> a + b);
```

Flow:

0 + 10 = 10

10 + 20 = 30

30 + 30 = 60

60 + 40 = 100

Result:

100

Think:

Many values

↓

combine

↓

One value

40. Stream pipeline complete example

```
List<Integer> numbers =
```

```
    Arrays.asList(10, 15, 20, 25, 30, 35);
```

Requirement:

Select numbers > 20, multiply by 2, sort them, and print.

Code:

```
numbers.stream()
```

```
    .filter(x -> x > 20)
```

```
    .map(x -> x * 2)
```

```
    .sorted()
```

```
    .forEach(System.out::println);
```

Let's trace.

Source

10 15 20 25 30 35

filter

$x \rightarrow x > 20$

Result:

25 30 35

map

$x \rightarrow x * 2$

Result:

50 60 70

sorted

50 60 70

forEach

Print:

50

60

70

41. Lazy evaluation

This is an important Stream concept.

Intermediate operations like:

`filter()`

`map()`

`sorted()`

are generally **lazy**.

Meaning they don't execute simply because you wrote them.

Example:

```
numbers.stream()
    .filter(x -> {
        System.out.println("Filtering " + x);
        return x > 20;
    });
```

If no terminal operation exists, stream processing doesn't actually complete.

Add:

```
.forEach(System.out::println);
```

Now processing happens.

So:

`stream()`

`filter()`

`map()`

↓

No terminal operation

↓

No actual pipeline execution

Then:

`forEach()`

↓

Terminal operation



Pipeline executes

42. Why lazy evaluation?

Performance kosam.

Example:

```
numbers.stream()  
    .filter(x -> x > 20)  
    .map(x -> x * 2)  
    .forEach(System.out::println);
```

Java doesn't necessarily do:

filter ALL



map ALL filtered



forEach ALL

Conceptually, stream processing can flow element-by-element through the pipeline.

For one element:

10



filter → false



discard

Next:

25



filter → true



map → 50

↓

forEach → print

This can avoid unnecessary work.

43. Streams do NOT modify original collection

Very important.

Suppose:

```
List<Integer> numbers =
```

```
    Arrays.asList(10, 20, 30);
```

Then:

```
numbers.stream()
```

```
    .map(x -> x * 2)
```

```
    .forEach(System.out::println);
```

Output:

20

40

60

But original:

numbers

is still:

10 20 30

Stream operations generally produce processed results rather than changing the source collection itself.

44. Stream can be used only once

This is an important interview point.

Example:

```
Stream<Integer> stream =
```

```
numbers.stream();
```

```
stream.forEach(System.out::println);
```

Then again:

```
stream.forEach(System.out::println);
```

will cause:

`IllegalStateException`

Because a stream is consumed after a terminal operation.

If you need another processing pipeline:

```
numbers.stream()
```

again create cheyyali.

45. Collection vs Stream

This is one of the most important distinctions.

Collection	Stream
Stores data	Processes data
Data structure	Processing abstraction
Can be reused	Usually one-time use
Example: <code>ArrayList</code>	Example: <code>list.stream()</code>
Data exists inside	Doesn't normally store data

Think:

`ArrayList`

↓

Storage

`Stream`

↓

Processing pipeline

Real-world:

Warehouse = Collection

Factory processing line = Stream

Warehouse lo products store chestam.

Factory line lo products ni:

filter

sort

transform

package

chestam.

46. Lambda + Functional Interface + Stream together

Now all three concepts connect.

Suppose:

List<Integer> numbers =

Arrays.asList(10, 15, 20, 25, 30);

Code:

numbers.stream()

.filter(x -> x > 20)

.map(x -> x * 2)

.forEach(x -> System.out.println(x));

Where is Lambda?

x -> x > 20

Lambda.

x -> x * 2

Lambda.

x -> System.out.println(x)

Lambda.

What functional interfaces are involved?

filter()

Takes a:

Predicate

because it needs:

true / false

map()

Takes a:

Function

because it needs:

input → output

forEach()

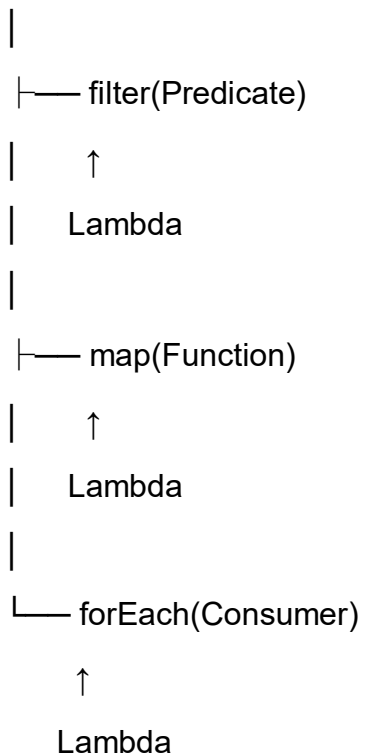
Takes a:

Consumer

because it performs an action without returning a value.

So:

Stream



This is the **core connection** of today's entire lesson.

47. method reference — bonus concept

You will frequently see:

```
System.out::println
```

instead of:

```
x -> System.out.println(x)
```

This is called a **method reference**.

Example:

```
numbers.forEach(x -> System.out.println(x));
```

can become:

```
numbers.forEach(System.out::println);
```

:: is method-reference syntax.

It basically means:

"Use this existing method as the behavior."

Examples:

```
String::length
```

```
System.out::println
```

```
Integer::parseInt
```

Method references are closely related to lambdas, but they are a separate syntax for referring to an existing method.

48. Practical example — Student filtering

Imagine:

```
class Student {
```

```
    String name;
```

```
    int marks;
```

```
    Student(String name, int marks) {
```

```
        this.name = name;
        this.marks = marks;
    }
}

Students:

List<Student> students = Arrays.asList(
    new Student("A", 85),
    new Student("B", 60),
    new Student("C", 92),
    new Student("D", 45)
);
```

Requirement:

Get students who scored above 70.

```
students.stream()
    .filter(s -> s.marks > 70)
    .forEach(s -> System.out.println(s.name));
```

Output:

A

C

Here:

`s -> s.marks > 70`

is Predicate behavior.

49. Get only student names

```
students.stream()
    .map(s -> s.name)
    .forEach(System.out::println);
```

Flow:

Student A → "A"

Student B → "B"

Student C → "C"

Student D → "D"

Here map() transforms:

Student → String

50. Filter + map

Students above 70 marks, then get names:

```
students.stream()
    .filter(s -> s.marks > 70)
    .map(s -> s.name)
    .forEach(System.out::println);
```

Flow:

Students

↓

filter marks > 70

↓

A, C

↓

map Student → name

↓

"A", "C"

↓

print

This type of code is extremely common in Java applications and interviews.

51. Collect filtered students into List

List<Student> toppers =

```
students.stream()
```

```
.filter(s -> s.marks > 70)
.collect(Collectors.toList());
```

Now:

toppers

↓

Student A

Student C

52. Find average marks

With streams:

double average =

```
students.stream()
    .mapToInt(s -> s.marks)
    .average()
    .orElse(0);
```

Here we introduced another concept:

mapToInt()

It converts objects to an IntStream.

Then:

average()

calculates average.

For beginner level, remember:

map() → normal Stream transformation

mapToInt() → converts values to IntStream

53. map() vs mapToInt()

Suppose:

```
students.stream()
    .map(s -> s.marks)
```

This gives a:

Stream<Integer>

But:

students.stream()

.mapToInt(s -> s.marks)

gives:

IntStream

Specialized primitive streams include:

IntStream

LongStream

DoubleStream

They provide numeric operations such as:

sum()

average()

max()

min()

54. Stream API important operations cheat sheet

Intermediate

filter()

map()

flatMap()

sorted()

distinct()

limit()

skip()

peek()

Terminal

forEach()

collect()
count()
reduce()
min()
max()
findFirst()
findAny()
anyMatch()
allMatch()
noneMatch()

For your current level, first master:

filter
map
sorted
distinct
limit
skip
forEach
collect
count
reduce
anyMatch
allMatch
noneMatch

55. findFirst()

Gets first element.

Optional<Integer> result =

 numbers.stream()


```
.filter(x -> x > 20)
    .findFirst();
```

If:

10 25 30 40

result:

25

Again Optional because matching element may not exist.

56. findAny()

Returns any matching element.

Optional<Integer> result =

```
    numbers.stream()
        .filter(x -> x > 20)
        .findAny();
```

Especially useful with parallel streams.

For sequential streams, it often looks similar to findFirst(), but their semantics are different.

57. peek() — debugging purpose

peek() is an intermediate operation.

Example:

```
numbers.stream()
    .filter(x -> x > 10)
    .peek(x -> System.out.println("After filter: " + x))
    .map(x -> x * 2)
    .forEach(System.out::println);
```

Useful for understanding/debugging a pipeline.

But don't normally use peek() as your main way of performing application side effects.

58. flatMap() — basic introduction

This is more advanced.

Suppose:

```
List<List<Integer>>
```

Example:

```
List<List<Integer>> numbers = Arrays.asList(  
    Arrays.asList(1, 2),  
    Arrays.asList(3, 4),  
    Arrays.asList(5, 6)  
);
```

We want:

```
1 2 3 4 5 6
```

map():

```
numbers.stream()  
    .map(list -> list)
```

still gives nested structure.

flatMap() flattens:

```
numbers.stream()  
    .flatMap(list -> list.stream())  
    .forEach(System.out::println);
```

Concept:

map

↓

```
List<List<Integer>>
```

flatMap

↓

```
List<Integer>
```

Think:

map() transforms, flatMap() transforms + flattens nested streams.

59. Parallel Stream

Java also has:

parallelStream()

Example:

```
numbers.parallelStream()
    .forEach(System.out::println);
```

This allows stream processing to use multiple threads.

But important:

Parallel stream doesn't automatically mean faster.

For small collections, parallel processing overhead may make it slower.

Also ordering and thread-safety considerations matter.

For now remember:

```
stream()
    → sequential processing
```

```
parallelStream()
    → parallel processing
```

60. Common mistakes beginners make

Mistake 1

Thinking Stream stores data.

✗ Wrong.

Collection → stores data

Stream → processes data

Mistake 2

Thinking lambda is a normal method.

Lambda is a concise way to represent behavior, typically targeting a functional interface.

Mistake 3

Thinking every interface can directly receive a lambda.

✗ No.

The target type must be a **functional interface**.

Mistake 4

Thinking filter() changes the original List.

✗ No.

It creates a stream of selected elements.

Mistake 5

Thinking map() filters.

✗ No.

filter → select

map → transform

Mistake 6

Thinking stream() modifies collection.

✗ No.

It creates a stream for processing.

Mistake 7

Forgetting terminal operation.

Example:

```
numbers.stream()
```

```
    .filter(x -> x > 10)
```

```
.map(x -> x * 2);
```

Pipeline is not consumed.

Usually you need something like:

```
.forEach(...)
```

or:

```
.collect(...)
```

or:

```
.count()
```

61. Most important interview questions

Q1. What is Lambda expression?

Answer:

A lambda expression is a concise way to represent an anonymous function/behavior in Java, usually used with functional interfaces.

Example:

```
(a, b) -> a + b
```

Q2. What is Functional Interface?

An interface containing exactly one abstract method is called a functional interface.

Example:

```
@FunctionalInterface
```

```
interface Calculator {
```

```
    int calculate(int a, int b);
```

```
}
```

Q3. Can functional interface have other methods?

Yes.

It can have:

- one abstract method

- any number of default methods
- any number of static methods

The rule is specifically about **abstract methods**.

For example:

```
@FunctionalInterface
```

```
interface Test {
```

```
    void run();
```

```
    default void display() {
```

```
        System.out.println("Hello");
```

```
    }
```

```
    static void show() {
```

```
        System.out.println("Show");
```

```
    }
```

```
}
```

Still functional interface because only one abstract method exists:

```
void run();
```

62. Q4. What is Stream API?

Stream API is a Java API used to process sequences of data in a declarative and functional style.

It supports operations such as:

filter

map

sort

reduce

collect

63. Q5. Is Stream a collection?

No.

Collection → stores data

Stream → processes data

64. Q6. Difference between filter() and map()?

filter()

↓

select elements based on condition

map()

↓

transform each element

Example:

`.filter(x -> x > 10)`

vs

`.map(x -> x * 2)`

65. Q7. What are intermediate operations?

Operations that return another stream.

Examples:

filter()

map()

sorted()

distinct()

limit()

skip()

They are generally lazy.

66. Q8. What are terminal operations?

Operations that produce a final result/action and consume the stream.

Examples:

`forEach()`

`collect()`

`count()`

`reduce()`

`min()`

`max()`

67. Q9. What is lazy evaluation?

Intermediate operations don't execute their processing immediately.

They wait until a terminal operation triggers the stream pipeline.

`filter()`

`map()`

↓

`wait`

↓

`forEach()`

↓

execution

68. Q10. Can a Stream be reused?

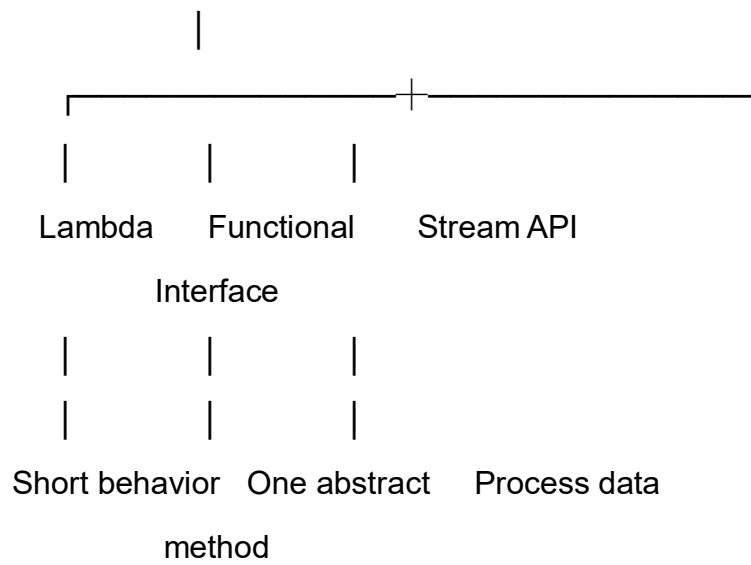
No.

After a terminal operation, the stream is consumed.

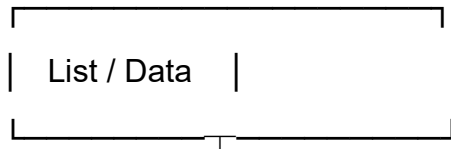
Create a new stream from the source if you need another pipeline.

69. The entire Day 38 in one picture

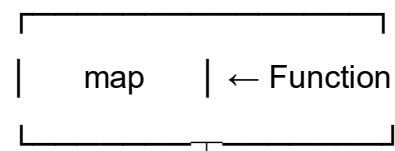
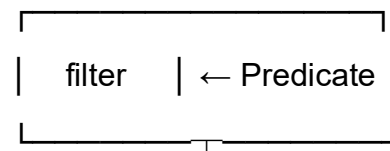
JAVA 8

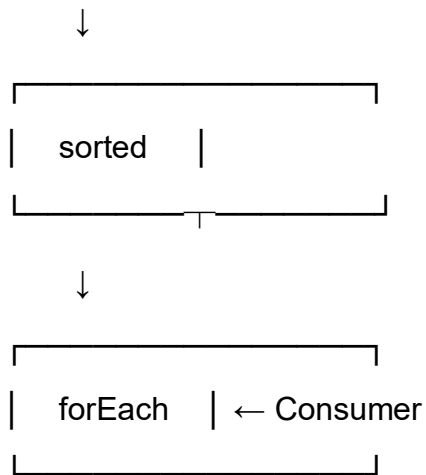


Work together



stream()





70. One final example combining everything

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class Main {

    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 15, 20, 25, 30, 35);

        List<Integer> result =
            numbers.stream()
                .filter(x -> x > 20)
                .map(x -> x * 2)
                .sorted()
                .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

```
}  
}
```

Output:

[50, 60, 70]

Full trace

Original:

[10, 15, 20, 25, 30, 35]

stream():

Create processing pipeline

filter():

$x > 20$

Result:

[25, 30, 35]

map():

$x * 2$

Result:

[50, 60, 70]

sorted():

[50, 60, 70]

collect():

Stream $\rightarrow$ List

Final:

[50, 60, 70]

And importantly:

$x \rightarrow x > 20$

is a lambda behaving like a **Predicate**.

$x \rightarrow x * 2$

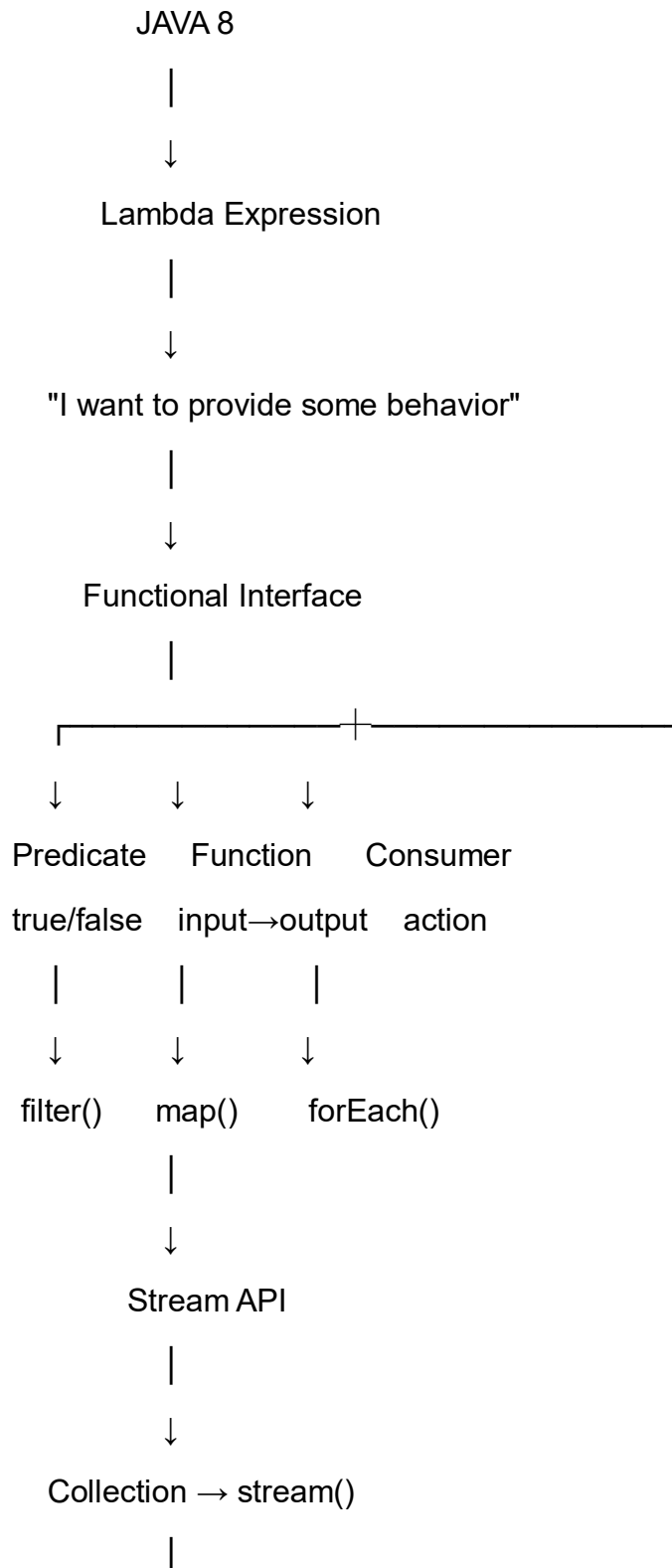
is a lambda behaving like a **Function**.

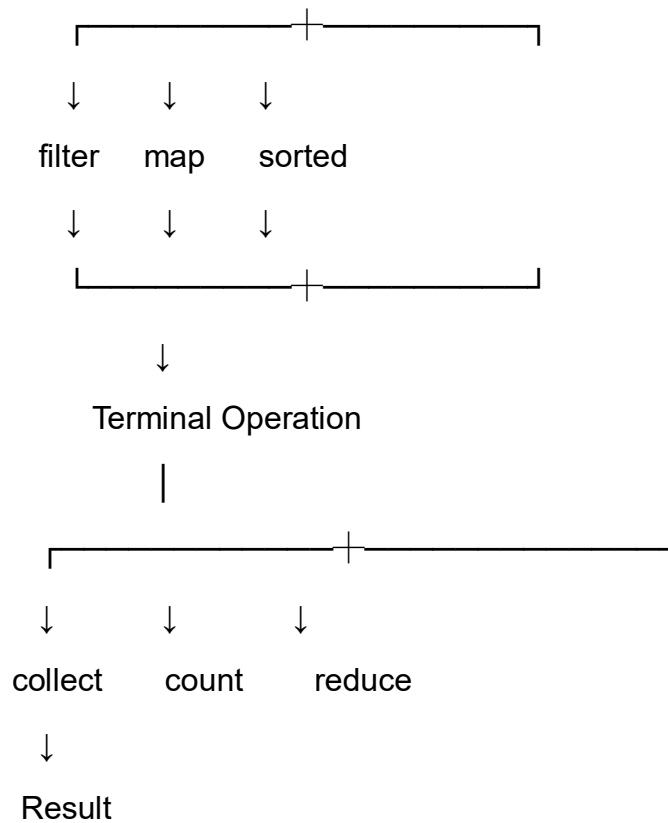
Collectors.toList()

collects the processed elements into a List.

□ Final memory map

Don't try to memorize everything separately. Remember this chain:





☆ If you remember only 10 things from Day 38:

1. **Lambda = short way to represent behavior.**
2. Lambda syntax: (parameters) -> expression.
3. **Functional Interface = exactly one abstract method.**
4. @FunctionalInterface tells the compiler to enforce that rule.
5. **Predicate = input → boolean.**
6. **Function = input → output.**
7. **Consumer = input → no return.**
8. **Supplier = no input → output.**
9. **Stream = data processing, not data storage.**
10. **filter() selects, map() transforms, terminal operation finishes the pipeline.**

The most important connection to remember is:

Lambda

↓

implements behavior of



Functional Interface



used heavily by



Stream API



filter → map → sort → collect

That connection is what makes Java 8 features much easier to understand rather than memorizing individual methods.